

Bridge Day 11 INSTRUCTOR KEY

While Loops, Update Order, Sentinels, and Termination

Learning sequence: Predict - Trace - Explain - Test - Interpret

Monday, July 20, 2026 • 50 points • longitudinal template 07242026_v1

Name		UIN		Section	
------	--	-----	--	---------	--

Daily target and independence boundary

Design loops whose state changes make termination demonstrable.

Allowed tools	Prior tools plus while loops and sentinel-controlled input
Support supplied	The sentinel, valid-data interval, and required summary are supplied.
Student decides	Loop pattern, update placement, validity branch, and the empty-data case.
Scaffold level	Termination evidence prompted; algorithm no longer sequenced line by line.

Evidence and scoring

Evidence	Points	Secure work shows
Integrated trace	12	intermediate state, condition results, exact output, and meaning
Diagnosis and repair	10	actual, intended, cause, repair, and a discriminating test
Complete program and tests	28	coherent executable logic, required output, assumptions, and defended tests

Task 1. Integrated trace - 12 points

Trace both parts of the compound condition before each pass and the state after each update.

```
level = 17
steps = 0

while level < 50 and steps < 4:
    level = level + 9
    steps = steps + 1
    print(level, steps)

print("final", level, steps)
```

Your task

1. Build one ledger with condition result, level, and steps before and after each pass.
2. Give every output line and explain which condition evidence proves termination.

Answer 1. Passes begin at (17,0), (26,1), (35,2), and (44,3), with both comparisons True each time. After updates the states are (26,1), (35,2), (44,3), and (53,4).

Answer 2. The loop prints 26 1; 35 2; 44 3; 53 4; then final 53 4. The next condition is False because $\text{level} < 50$ is False and $\text{steps} < 4$ is also False. Either false operand is enough to stop an and condition.

Task 2. Diagnose and repair - 10 points

This loop should read integers until -1 and count nonnegative entries, but it can repeat forever.

```
reading = int(input())
valid_count = 0

while reading != -1:
    if reading >= 0:
        valid_count = valid_count + 1

print(valid_count)
```

Your task

1. Explain the infinite-loop condition using the unchanged state, not just the phrase missing update.
2. Write the minimal repair and describe the final false condition for inputs 5, 8, -1.

Answer 1. If the first reading is not -1, reading never changes in the body. Therefore reading != -1 remains True forever, and the same value is repeatedly counted when it is nonnegative.

Answer 2. Add reading = int(input()) as the final statement in the loop body. For inputs 5, 8, -1, valid_count becomes 2; after -1 is stored, reading != -1 is False and the loop stops before counting the sentinel.

Task 3. Construct a complete program - 28 points

Program specification

- Read integer readings until the sentinel -1 is entered.
- Values from 0 through 100 inclusive are valid. Any other non-sentinel value is ignored and must print Ignored.
- For valid values, compute count, total, and the number at or above 70.
- After termination, print count, total, and high_count on separate lines. Then print the average, or print No data when count is zero.

Required planning evidence

1. State the loop-controlling variable and show where it changes so that every path either terminates or reaches another input operation.

Program workspace**Model program**

```
reading = int(input())
count = 0
total = 0
high_count = 0

while reading != -1:
    if 0 <= reading <= 100:
        count = count + 1
        total = total + reading
        if reading >= 70:
            high_count = high_count + 1
    else:
        print("Ignored")
        reading = int(input())

print(count)
print(total)
print(high_count)
if count == 0:
    print("No data")
else:
    print(total / count)
```

Reasoning: The first input occurs before the condition, and exactly one new input occurs at the bottom of every completed pass. This guarantees progress toward a future sentinel regardless of whether a value is valid.

Reasoning: The average branch is after the loop. It protects division by zero while preserving all requested summary output.

Task 3 continued. Test and defend - included in 28 points

Continue code if needed.

Expected tests and results

1. Inputs 70, 101, 20, -1 print Ignored during processing, then 2, 90, 1, and 45.0 on separate summary lines.
2. Immediate -1 prints 0, 0, 0, and No data.

Required tests, expected results, and brief defense

Scoring guidance: 3 points termination plan; 4 initialization/input; 6 sentinel loop and update; 5 validity handling; 4 summaries; 3 empty case; 3 tests/explanation.

Bridge Day 11 INSTRUCTOR KEY

VIP Evidence Handoff

Learning sequence: Predict - Trace - Explain - Test - Interpret

Contract 07a04826c975 • longitudinal template 07242026_v1

Why engineers use this page

Complete the same while-loop cells with less planning support and defend the empty or already-finished case.

Decision boundary: Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

Construct readiness before independent work

New authoring tools: for loop, while loop, range call, loop state update, termination condition, list literal as collector, append call

Carried authoring tools: assignment, print call, arithmetic expression, modulo, input call, int conversion, f string, comparison expression, if elif else

Supplied-code exposure only: list index read

An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

Notebook cells and task constructs

Activity	Work	Stable cells	Required authoring constructs
18.19	Countdown To Multiple	18-19-work / 18-19-transfer / 18-19-cold	arithmetic expression, while loop
18.21	Cooldown Sequence	18-21-work / 18-21-transfer / 18-21-cold	append call, arithmetic expression, while loop
18.22	Rounds To Clear Queue	18-22-work / 18-22-transfer / 18-22-cold	arithmetic expression, f string, input call, int conversion, while loop

Readiness evidence

1. Predict whether each loop executes zero, one, or multiple times for its transfer case before running it.

Model response: Predictions must follow the actual transfer inputs and initial condition. Credit the reasoning even if the initial prediction is later revised from output evidence.

2. Identify where output or list append must occur relative to the update.

Model response: 18.21 appends the updated state so the final nonpositive value is retained; 18.22 increments once per completed reduction round; 18.19 prints only after the first matching value is reached.

Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** _____ **My help trigger:** _____

Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

Evidence field	Record
Prediction	A specific expected state or output before execution.
Observed Result	The actual state or output, including a value or exact text.
Revision Reason	The defect or mismatch and the change that addresses it.
Engineering Meaning	How the evidence changes or supports the engineering decision.
Units Or Not Applicable	The physical unit, or the evidence type when no unit applies.
Assumption	One condition accepted as true for this model or rule.
Model Limit	One situation the result does not justify or predict.
Next Test	A concrete boundary, invalid, or alternate case with an expected result.
Help Trigger	The exact unresolved point that would trigger a targeted question.
Minutes Used	Approximate minutes from first prediction through revision.

Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.